Bref historique

- Années 70 : développement du langage de preuve de programmes ML (Meta Language) (R. Milner).

Bref historique

- Années 70 : développement du langage de preuve de programmes ML (Meta Language) (R. Milner).
- 1985 : première implémentation de Caml (Categorical Abstract Machine Language) à l'INRIA (organisme de recherche français).

Bref historique

- Années 70 : développement du langage de preuve de programmes ML (Meta Language) (R. Milner).
- 1985 : première implémentation de Caml (Categorical Abstract Machine Language) à l'INRIA (organisme de recherche français).
- 1996 : première version d'OCaml (Objective Caml) par X. Leroy.

Bref historique

- Années 70 : développement du langage de preuve de programmes ML (Meta Language) (R. Milner).
- 1985 : première implémentation de Caml (Categorical Abstract Machine Language) à l'INRIA (organisme de recherche français).
- 1996 : première version d'OCaml (Objective Caml) par X. Leroy.
- 2005 : première version de F#, variante d'OCaml développée par Microsoft.

Bref historique

- Années 70 : développement du langage de preuve de programmes ML (Meta Language) (R. Milner).
- 1985 : première implémentation de Caml (Categorical Abstract Machine Language) à l'INRIA (organisme de recherche français).
- 1996 : première version d'OCaml (Objective Caml) par X. Leroy.
- 2005 : première version de F#, variante d'OCaml développée par Microsoft.
- 2016 : première version de Reason, variante d'OCaml développée par Facebook.

Bref historique

- Années 70 : développement du langage de preuve de programmes ML (Meta Language) (R. Milner).
- 1985 : première implémentation de Caml (Categorical Abstract Machine Language) à l'INRIA (organisme de recherche français).
- 1996 : première version d'OCaml (Objective Caml) par X. Leroy.
- 2005 : première version de F#, variante d'OCaml développée par Microsoft.
- 2016 : première version de Reason, variante d'OCaml développée par Facebook.
- 2022 : OCaml version 5.0.0

Quelques caractéristiques

- OCaml est un langage de programmation fonctionnelle, les fonctions sont au cœur de ce paradigme de programmation.

Quelques caractéristiques

- OCaml est un langage de programmation fonctionnelle, les fonctions sont au cœur de ce paradigme de programmation.
- Les variables sont *non modifiables* ; en conséquence, la récursivité est fondamentale car l'écriture de boucles devient impossible. La motivation est de produire un code plus lisible, plus facile à maintenir et moins sujet aux bugs.

Quelques caractéristiques

- OCaml est un langage de programmation fonctionnelle, les fonctions sont au cœur de ce paradigme de programmation.
- Les variables sont *non modifiables* ; en conséquence, la récursivité est fondamentale car l'écriture de boucles devient impossible. La motivation est de produire un code plus lisible, plus facile à maintenir et moins sujet aux bugs.
- OCaml est typé statiquement, une variable ne peut pas changer de type au cours de l'exécution. De plus, les erreurs de type seront systématiquement détectées à la compilation.

Quelques caractéristiques

- OCaml est un langage de programmation fonctionnelle, les fonctions sont au cœur de ce paradigme de programmation.
- Les variables sont *non modifiables* ; en conséquence, la récursivité est fondamentale car l'écriture de boucles devient impossible. La motivation est de produire un code plus lisible, plus facile à maintenir et moins sujet aux bugs.
- OCaml est typé statiquement, une variable ne peut pas changer de type au cours de l'exécution. De plus, les erreurs de type seront systématiquement détectées à la compilation.
- Le type des variables n'a pas besoin d'être précisé, il sera automatiquement détecté par le compilateur grâce à un procédé appelé inférence de type.

Quelques caractéristiques

- OCaml est un langage de programmation fonctionnelle, les fonctions sont au cœur de ce paradigme de programmation.
- Les variables sont *non modifiables* ; en conséquence, la récursivité est fondamentale car l'écriture de boucles devient impossible. La motivation est de produire un code plus lisible, plus facile à maintenir et moins sujet aux bugs.
- OCaml est typé statiquement, une variable ne peut pas changer de type au cours de l'exécution. De plus, les erreurs de type seront systématiquement détectées à la compilation.
- Le type des variables n'a pas besoin d'être précisé, il sera automatiquement détecté par le compilateur grâce à un procédé appelé inférence de type.
- OCaml est un langage compilé, cependant un environnement interactif `utop` est disponible.

Quelques caractéristiques

- OCaml est un langage de programmation fonctionnelle, les fonctions sont au cœur de ce paradigme de programmation.
- Les variables sont *non modifiables* ; en conséquence, la récursivité est fondamentale car l'écriture de boucles devient impossible. La motivation est de produire un code plus lisible, plus facile à maintenir et moins sujet aux bugs.
- OCaml est typé statiquement, une variable ne peut pas changer de type au cours de l'exécution. De plus, les erreurs de type seront systématiquement détectées à la compilation.
- Le type des variables n'a pas besoin d'être précisé, il sera automatiquement détecté par le compilateur grâce à un procédé appelé inférence de type.
- OCaml est un langage compilé, cependant un environnement interactif `utop` est disponible.
- La gestion de la mémoire est automatique (via un ramasse-miettes, en anglais *garbage collector*).

Fonctions sur les entiers et les flottants

1. Fonction qui, pour un entier n, renvoie $n(n-1)$.

```
1  let f n = n * (n-1);;
```

Fonctions sur les entiers et les flottants

1. Fonction qui, pour un entier n, renvoie $n(n-1)$.

```
1  let f n = n * (n-1);;
```

- ⚠ Dans le paradigme fonctionnel, on écrit des expressions et pas des instructions (paradigme impératif). Contrairement à une instruction, une expression est évaluée et renvoie toujours une valeur.

Fonctions sur les entiers et les flottants

1. Fonction qui, pour un entier n, renvoie $n(n-1)$.

```
1  let f n = n * (n-1);;
```

- ⚠ Dans le paradigme fonctionnel, on écrit des expressions et pas des instructions (paradigme impératif). Contrairement à une instruction, une expression est évaluée et renvoie toujours une valeur.
- On remarquera la proximité avec $f : n \mapsto n(n-1)$ (et l'absence de `return`).

Fonctions sur les entiers et les flottants

1. Fonction qui, pour un entier n, renvoie $n(n-1)$.

```
1  let f n = n * (n-1);;
```

- ⚠ Dans le paradigme fonctionnel, on écrit des expressions et pas des instructions (paradigme impératif). Contrairement à une instruction, une expression est évaluée et renvoie toujours une valeur.
- On remarquera la proximité avec $f : n \mapsto n(n-1)$ (et l'absence de `return`).
- Les opérateurs `*`, `-` portent sur les entiers et permettent d'inférer le type de l'argument et du résultat.

Fonctions sur les entiers et les flottants

1. Fonction qui, pour un entier n, renvoie $n(n-1)$.

```
1  let f n = n * (n-1);;
```

- ⚠ Dans le paradigme fonctionnel, on écrit des expressions et pas des instructions (paradigme impératif). Contrairement à une instruction, une expression est évaluée et renvoie toujours une valeur.
- On remarquera la proximité avec $f : n \mapsto n(n-1)$ (et l'absence de `return`).
- Les opérateurs `*`, `-` portent sur les entiers et permettent d'inférer le type de l'argument et du résultat.
- Pour calculer $f(5)$: `f 5 ;;`

Fonctions sur les entiers et les flottants

1. Fonction qui, pour un entier n, renvoie $n(n-1)$.

```
let f n = n * (n-1);;
```

   - ⚠ Dans le paradigme fonctionnel, on écrit des expressions et pas des instructions (paradigme impératif). Contrairement à une instruction, une expression est évaluée et renvoie toujours une valeur.
   - On remarquera la proximité avec $f : n \mapsto n(n-1)$ (et l'absence de `return`).
   - Les opérateurs `*`, `-` portent sur les entiers et permettent d'inférer le type de l'argument et du résultat.
   - Pour calculer $f(5)$: `f 5 ;;`

2. Fonction qui, pour un flottant x, renvoie $x^2 - 3x + 7$.

```
let g x = x**2. -. 3.0*.x +. 7.0 ;;
```

Fonctions sur les entiers et les flottants

1. Fonction qui, pour un entier n, renvoie $n(n-1)$.

```
let f n = n * (n-1);;
```

- ⚠ Dans le paradigme fonctionnel, on écrit des expressions et pas des instructions (paradigme impératif). Contrairement à une instruction, une expression est évaluée et renvoie toujours une valeur.
- On remarquera la proximité avec $f : n \mapsto n(n-1)$ (et l'absence de `return`).
- Les opérateurs `*`, `-` portent sur les entiers et permettent d'inférer le type de l'argument et du résultat.
- Pour calculer $f(5)$: `f 5 ;;`

2. Fonction qui, pour un flottant x, renvoie $x^2 - 3x + 7$.

```
let g x = x**2. -. 3.0*.x +. 7.0 ;;
```

- Les opérateurs `+.`, `*.` et `-.` concernent les flottants.

Fonctions sur les entiers et les flottants

1. Fonction qui, pour un entier n, renvoie $n(n-1)$.

```
let f n = n * (n-1);;
```

- ⚠ Dans le paradigme fonctionnel, on écrit des expressions et pas des instructions (paradigme impératif). Contrairement à une instruction, une expression est évaluée et renvoie toujours une valeur.
- On remarquera la proximité avec $f : n \mapsto n(n-1)$ (et l'absence de `return`).
- Les opérateurs `*`, `-` portent sur les entiers et permettent d'inférer le type de l'argument et du résultat.
- Pour calculer $f(5)$: `f 5 ;;`

2. Fonction qui, pour un flottant x, renvoie $x^2 - 3x + 7$.

```
let g x = x**2. -. 3.0*.x +. 7.0 ;;
```

- Les opérateurs `+.`, `*.` et `-.` concernent les flottants.
- L'exponentiation est `**` (sur les flottants uniquement).

Polymorphisme, booléens, conditionnelle

1. Fonction qui renvoie `true` lorsque deux des trois arguments sont égaux.

```
1  let deux_egaux a b c = (a=b) || (b=c) || (a=c);;
```

Polymorphisme, booléens, conditionnelle

1. Fonction qui renvoie `true` lorsque deux des trois arguments sont égaux.

```
1  let deux_egaux a b c = (a=b) || (b=c) || (a=c);;
```

   - Les opérateurs logiques sont `&&`, `||` et `not`.

Polymorphisme, booléens, conditionnelle

1. Fonction qui renvoie `true` lorsque deux des trois arguments sont égaux.

```
1  let deux_egaux a b c = (a=b) || (b=c) || (a=c);;
```

   - Les opérateurs logiques sont `&&`, `||` et `not`.
   - Ici l'inférence de type ne permet pas de déterminer le type des arguments, on dit que la fonction est polymorphe.

Polymorphisme, booléens, conditionnelle

1. Fonction qui renvoie `true` lorsque deux des trois arguments sont égaux.

```
1  let deux_egaux a b c = (a=b) || (b=c) || (a=c);;
```

   - Les opérateurs logiques sont `&&`, `||` et `not`.
   - Ici l'inférence de type ne permet pas de déterminer le type des arguments, on dit que la fonction est polymorphe.
   - L'appel s'effectue en donnant les paramètres séparés par des espaces : `deux_egaux 4 6 4;;`

Polymorphisme, booléens, conditionnelle

1. Fonction qui renvoie `true` lorsque deux des trois arguments sont égaux.

```
let deux_egaux a b c = (a=b) || (b=c) || (a=c);;
```

   - Les opérateurs logiques sont `&&`, `||` et `not`.
   - Ici l'inférence de type ne permet pas de déterminer le type des arguments, on dit que la fonction est polymorphe.
   - L'appel s'effectue en donnant les paramètres séparés par des espaces : `deux_egaux 4 6 4;;`

2. Terme suivant de la suite de Syracuse :

```
let syracuse n =
  if n mod 2 = 0 then n/2 else 3*n+1;;
```

Polymorphisme, booléens, conditionnelle

1. Fonction qui renvoie `true` lorsque deux des trois arguments sont égaux.

```
let deux_egaux a b c = (a=b) || (b=c) || (a=c);;
```

   - Les opérateurs logiques sont `&&`, `||` et `not`.
   - Ici l'inférence de type ne permet pas de déterminer le type des arguments, on dit que la fonction est polymorphe.
   - L'appel s'effectue en donnant les paramètres séparés par des espaces : `deux_egaux 4 6 4;;`

2. Terme suivant de la suite de Syracuse :

```
let syracuse n =
  if n mod 2 = 0 then n/2 else 3*n+1;;
```

   - On notera la construction `if ... then ... else`.

Polymorphisme, booléens, conditionnelle

1. Fonction qui renvoie `true` lorsque deux des trois arguments sont égaux.

```
let deux_egaux a b c = (a=b) || (b=c) || (a=c);;
```

   - Les opérateurs logiques sont `&&`, `||` et `not`.
   - Ici l'inférence de type ne permet pas de déterminer le type des arguments, on dit que la fonction est polymorphe.
   - L'appel s'effectue en donnant les paramètres séparés par des espaces : `deux_egaux 4 6 4;;`

2. Terme suivant de la suite de Syracuse :

```
let syracuse n =
  if n mod 2 = 0 then n/2 else 3*n+1;;
```

   - On notera la construction `if ... then ... else`.
   - Attention, le test d'égalité est le `=` simple.

Polymorphisme, booléens, conditionnelle

1. Fonction qui renvoie `true` lorsque deux des trois arguments sont égaux.

```
let deux_egaux a b c = (a=b) || (b=c) || (a=c);;
```

   - Les opérateurs logiques sont `&&`, `||` et `not`.
   - Ici l'inférence de type ne permet pas de déterminer le type des arguments, on dit que la fonction est polymorphe.
   - L'appel s'effectue en donnant les paramètres séparés par des espaces : `deux_egaux 4 6 4;;`

2. Terme suivant de la suite de Syracuse :

```
let syracuse n =
  if n mod 2 = 0 then n/2 else 3*n+1;;
```

   - On notera la construction `if ... then ... else`.
   - Attention, le test d'égalité est le `=` simple.
   - Le modulo s'obtient avec `mod`.

Récursivité : exemple 1

Fonction qui calcule la somme des n premiers carrés.

```
1  let rec somme_carre n =
2    if n=0 then 0 else n*n + somme_carre (n-1);;
```

Récursivité : exemple 1

Fonction qui calcule la somme des n premiers carrés.

```
1  let rec somme_carre n =
2    if n=0 then 0 else n*n + somme_carre (n-1);;
```

- On notera le mot-clé `rec` pour préciser que la fonction est récursive.

Récursivité : exemple 1

Fonction qui calcule la somme des n premiers carrés.

```
1  let rec somme_carre n =
2    if n=0 then 0 else n*n + somme_carre (n-1);;
```

- On notera le mot-clé `rec` pour préciser que la fonction est récursive.
- Les parenthèses autour de `n-1` permettent d'éviter la confusion avec `(somme_carre n) - 1` (et donc d'avoir une récursion infinie).

Récursivité : exemple 2

Fonction qui compte à rebours depuis n et affiche « Partez ! »

```
1  let rec compte_rebours n =
2    if n=0 then print_endline "Partez" else (
3      print_int n;
4      print_endline "";
5      compte_rebours (n-1) );;
```

Récursivité : exemple 2

Fonction qui compte à rebours depuis n et affiche « Partez ! »

```
1  let rec compte_rebours n =
2    if n=0 then print_endline "Partez" else (
3      print_int n;
4      print_endline "";
5      compte_rebours (n-1) );;
```

- Regroupement des expressions de la branche `else` dans un bloc délimité par `( )`.
  On peut de façon équivalente délimiter par `begin` et `end`.

Récursivité : exemple 2

Fonction qui compte à rebours depuis n et affiche « Partez ! »

```
1  let rec compte_rebours n =
2    if n=0 then print_endline "Partez" else (
3      print_int n;
4      print_endline "";
5      compte_rebours (n-1) );;
```

- Regroupement des expressions de la branche `else` dans un bloc délimité par `( )`.
  On peut de façon équivalente délimiter par `begin` et `end`.
- Les résultats des affichages sont aussi des expressions et renvoient donc une valeur. Ce sont des valeurs de type `unit` (et ont toutes la même valeur : `()`).

Récursivité : exemple 2

Fonction qui compte à rebours depuis n et affiche « Partez ! »

```
1  let rec compte_rebours n =
2    if n=0 then print_endline "Partez" else (
3      print_int n;
4      print_endline "";
5      compte_rebours (n-1) );;
```

- Regroupement des expressions de la branche `else` dans un bloc délimité par `( )`.
  On peut de façon équivalente délimiter par `begin` et `end`.
- Les résultats des affichages sont aussi des expressions et renvoient donc une valeur. Ce sont des valeurs de type `unit` (et ont toutes la même valeur : `()`).
- Le `;` est le séquenceur : il sépare deux expressions, ignore la valeur de la première (qui doit être de type `unit`) et évalue la seconde.

À retenir !

- Un programme OCaml consiste en l'évaluation d'une ou plusieurs expressions, **il n'y a pas d'instructions**.

À retenir !

- Un programme OCaml consiste en l'évaluation d'une ou plusieurs expressions, **il n'y a pas d'instructions**.
  Une structure `if` ... `then` ... `else` s'évalue en une valeur. En impératif, ce sont des instructions conditionnelles.

À retenir !

- Un programme OCaml consiste en l'évaluation d'une ou plusieurs expressions, **il n'y a pas d'instructions**.
  Une structure `if` ... `then` ... `else` s'évalue en une valeur. En impératif, ce sont des instructions conditionnelles.
- Les types n'ont pas à être spécifiés, ils sont déterminés automatiquement par le mécanisme d'inférence.
  Les opérateurs sont donc associés à des types précis, par exemple + est l'addition sur les entiers, `+.` est l'addition sur les flottants, `@` est l'opérateur de concaténation des listes, `^` est l'opérateur de concaténation des chaînes de caractères.

À retenir !

- Un programme OCaml consiste en l'évaluation d'une ou plusieurs expressions, **il n'y a pas d'instructions**.
  Une structure `if ... then ... else` s'évalue en une valeur. En impératif, ce sont des instructions conditionnelles.
- Les types n'ont pas à être spécifiés, ils sont déterminés automatiquement par le mécanisme d'inférence.
  Les opérateurs sont donc associés à des types précis, par exemple + est l'addition sur les entiers, `+.` est l'addition sur les flottants, `@` est l'opérateur de concaténation des listes, `^` est l'opérateur de concaténation des chaînes de caractères.
- Toute expression a une valeur, la détermination de cette valeur s'appelle l'évaluation.

Types de base

Type	Opérations	Commentaires

Types de base

Type	Opérations	Commentaires
`int`	`+`, `-`, `*`, `/`, `mod`, `abs`	Entiers signés codés sur 63 bits (sur machine 64 bits), valeurs dans $[\![-2^{62}; 2^{62}-1]\!]$. Dépassement silencieux, exception `Division_by_zero`.

Types de base

Type	Opérations	Commentaires
`int`	`+`, `-`, `*`, `/`, `mod`, `abs`	Entiers signés codés sur 63 bits (sur machine 64 bits), valeurs dans $[\![-2^{62}; 2^{62}-1]\!]$. Dépassement silencieux, exception `Division_by_zero`.
`float`	`+.`, `-.`, `*.`, `/.`, `**`	Correspond au type double de la norme IEEE-754. Fonctions mathématiques usuelles ($\sin, \exp, \ldots$)

Types de base

Type	Opérations	Commentaires
`int`	`+`, `-`, `*`, `/`, `mod`, `abs`	Entiers signés codés sur 63 bits (sur machine 64 bits), valeurs dans $[\![-2^{62}; 2^{62}-1]\!]$. Dépassement silencieux, exception `Division_by_zero`.
`float`	`+.`, `-.`, `*.`, `/.`, `**`	Correspond au type double de la norme IEEE-754. Fonctions mathématiques usuelles ($\sin, \exp, \ldots$)
`bool`	`&&`, `||`, `not`	Évaluations paresseuses (court-circuit). Les valeurs sont notées `true` et `false`.

Types de base

Type	Opérations	Commentaires
`int`	`+`, `-`, `*`, `/`, `mod`, `abs`	Entiers signés codés sur 63 bits (sur machine 64 bits), valeurs dans $[\![-2^{62}; 2^{62}-1]\!]$. Dépassement silencieux, exception `Division_by_zero`.
`float`	`+.`, `-.`, `*.`, `/.`, `**`	Correspond au type double de la norme IEEE-754. Fonctions mathématiques usuelles ($\sin, \exp, \ldots$)
`bool`	`&&`, `||`, `not`	Évaluations paresseuses (court-circuit). Les valeurs sont notées `true` et `false`.
`char`		Se note entre apostrophes (`'` et `'`). Les `char` sont comparables (ordre du code ASCII).

Types de base

Type	Opérations	Commentaires
`int`	`+`, `-`, `*`, `/`, `mod`, `abs`	Entiers signés codés sur 63 bits (sur machine 64 bits), valeurs dans $[\![-2^{62}; 2^{62}-1]\!]$. Dépassement silencieux, exception `Division_by_zero`.
`float`	`+.`, `-.`, `*.`, `/.`, `**`	Correspond au type double de la norme IEEE-754. Fonctions mathématiques usuelles ($\sin, \exp, \ldots$)
`bool`	`&&`, `||`, `not`	Évaluations paresseuses (court-circuit). Les valeurs sont notées `true` et `false`.
`char`		Se note entre apostrophes (`'` et `'`). Les `char` sont comparables (ordre du code ASCII).
`string`	`^`, `s.[i]`, `String.length`	Immuabilité. Concaténation de deux chaînes : `"Bon"^"jour"`. Accès au i-ième caractère avec `s.[i]`

Types de base

Type	Opérations	Commentaires
`int`	`+`, `-`, `*`, `/`, `mod`, `abs`	Entiers signés codés sur 63 bits (sur machine 64 bits), valeurs dans $[\![-2^{62}; 2^{62}-1]\!]$. Dépassement silencieux, exception `Division_by_zero`.
`float`	`+.`, `-.`, `*.`, `/.`, `**`	Correspond au type double de la norme IEEE-754. Fonctions mathématiques usuelles ($\sin, \exp, \ldots$)
`bool`	`&&`, `||`, `not`	Évaluations paresseuses (court-circuit). Les valeurs sont notées `true` et `false`.
`char`		Se note entre apostrophes (`'` et `'`). Les `char` sont comparables (ordre du code ASCII).
`string`	`^`, `s.[i]`, `String.length`	Immuabilité. Concaténation de deux chaînes : `"Bon"^"jour"`. Accès au i-ième caractère avec `s.[i]`

- Le type `unit` possède une seule valeur notée `()` à rapprocher du type `void` du C. Un affichage renvoie `()`.

Types de base

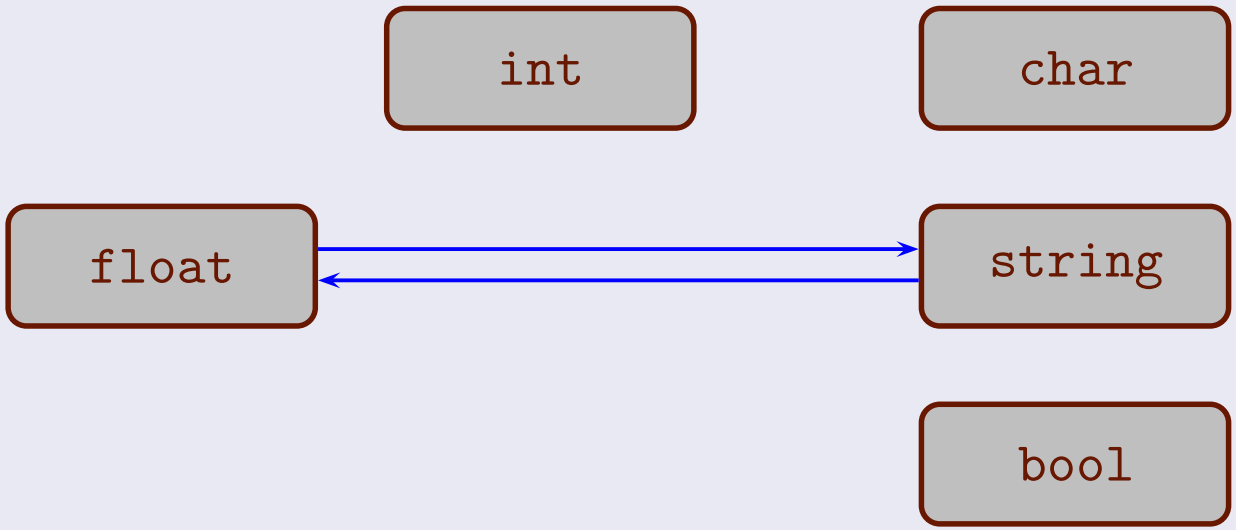
Type	Opérations	Commentaires
`int`	`+`, `-`, `*`, `/`, `mod`, `abs`	Entiers signés codés sur 63 bits (sur machine 64 bits), valeurs dans $[\![-2^{62}; 2^{62}-1]\!]$. Dépassement silencieux, exception `Division_by_zero`.
`float`	`+.`, `-.`, `*.`, `/.`, `**`	Correspond au type double de la norme IEEE-754. Fonctions mathématiques usuelles ($\sin, \exp, \ldots$)
`bool`	`&&`, `||`, `not`	Évaluations paresseuses (court-circuit). Les valeurs sont notées `true` et `false`.
`char`		Se note entre apostrophes (' et '). Les `char` sont comparables (ordre du code ASCII).
`string`	`^`, `s.[i]`, `String.length`	Immuabilité. Concaténation de deux chaînes : `"Bon"^"jour"`. Accès au i-ième caractère avec `s.[i]`

- Le type `unit` possède une seule valeur notée `()` à rapprocher du type `void` du C. Un affichage renvoie `()`.
- Les opérateurs de comparaison (`=`, `<>`, `>`, `>=`, `<`, `<=`) sont polymorphes mais

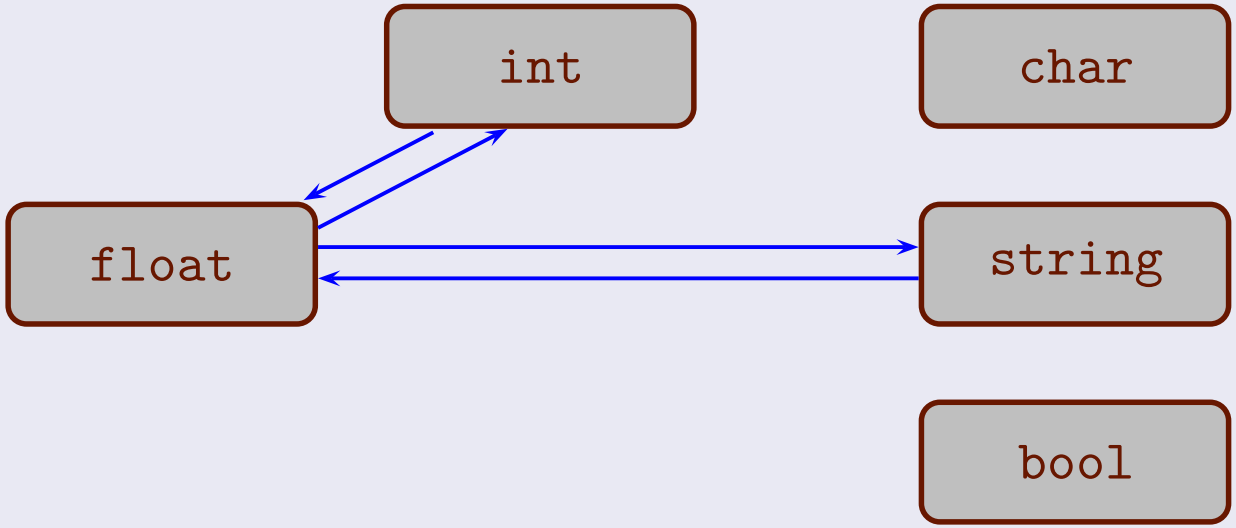
Conversion de types



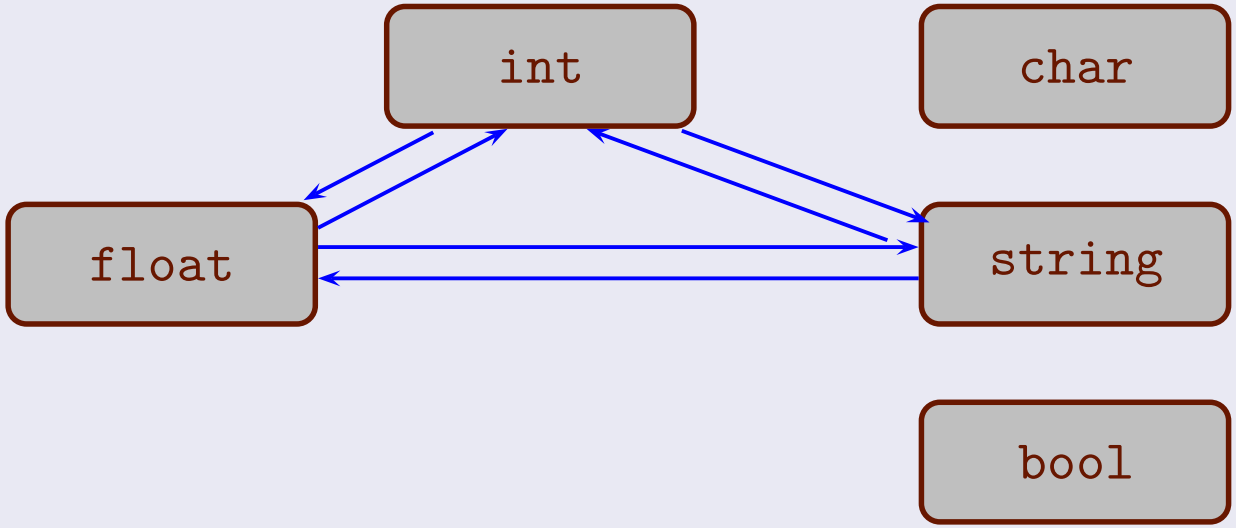
Conversion de types



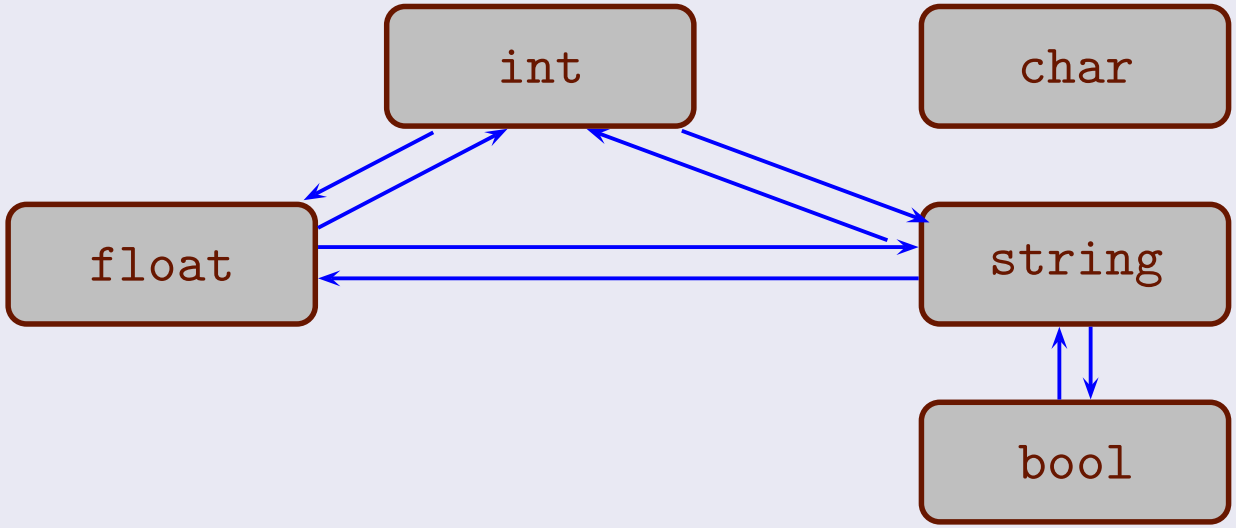
Conversion de types



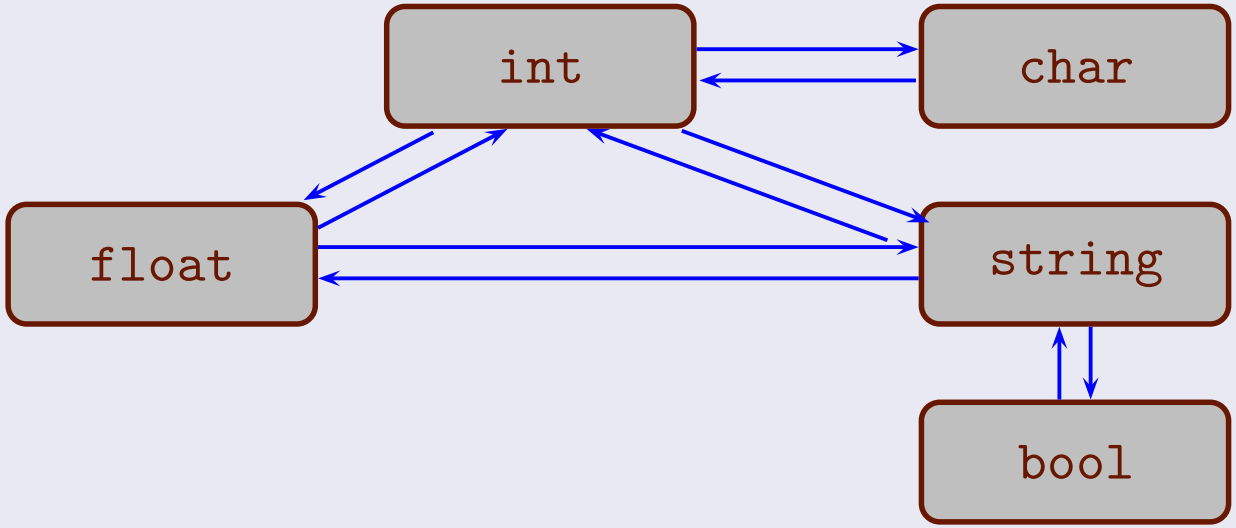
Conversion de types



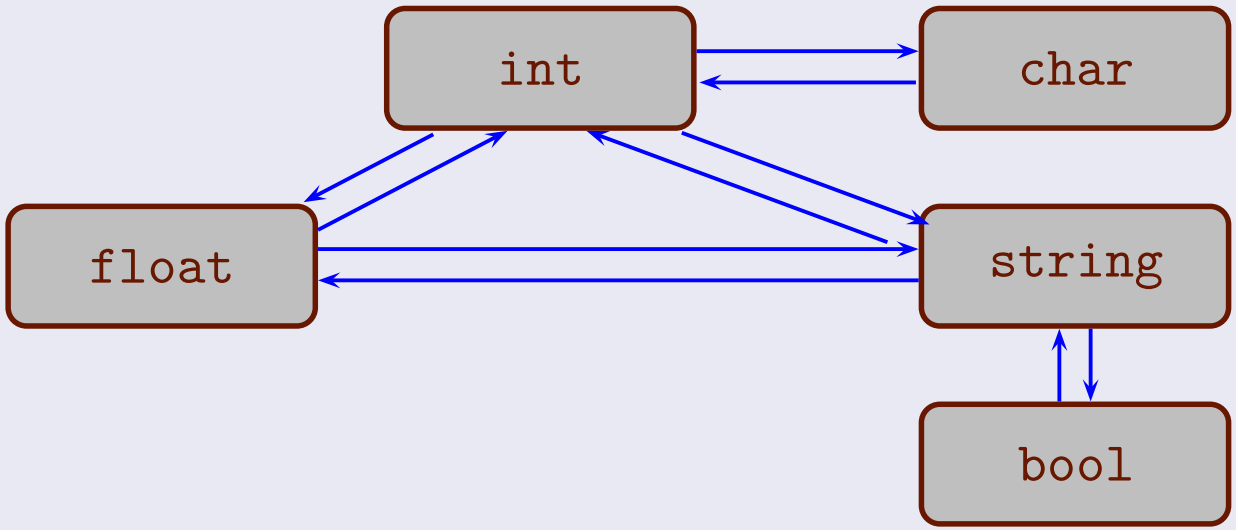
Conversion de types



Conversion de types

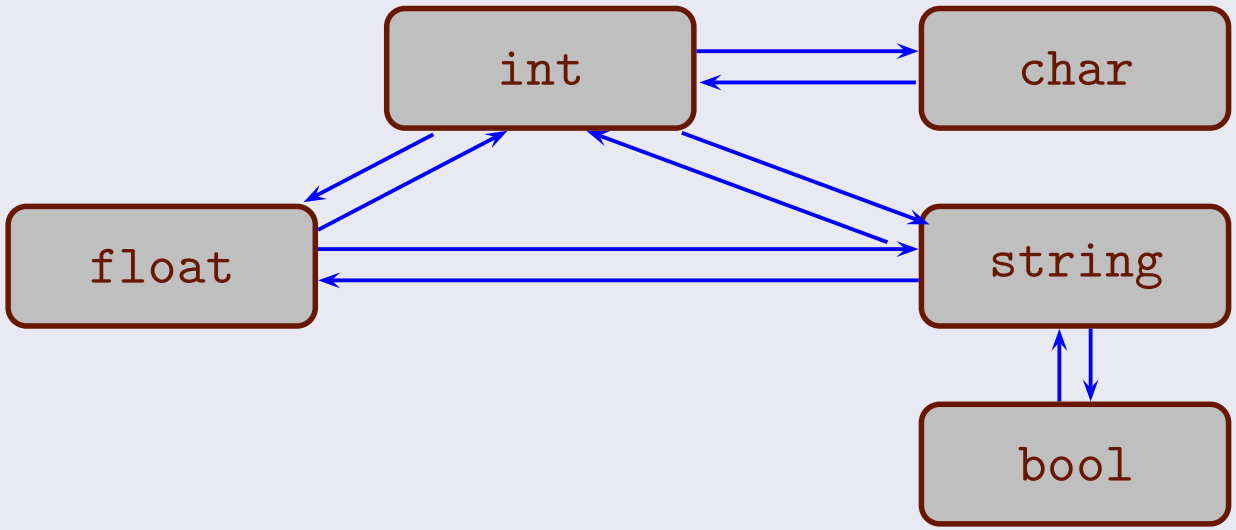


Conversion de types



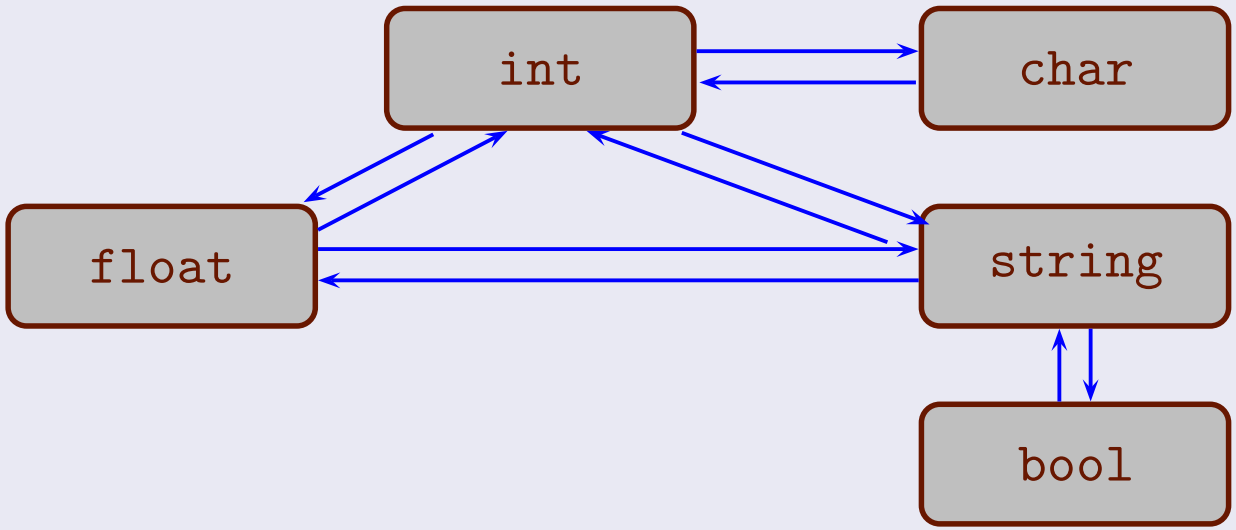
- Les fonctions de conversion sont de la forme `<type_destination>_of_<type_source>`, par exemple `string_of_float` (convertit un flottant en chaîne).

Conversion de types



- Les fonctions de conversion sont de la forme `<type_destination>_of_<type_source>`, par exemple `string_of_float` (convertit un flottant en chaîne).
- Il n'y a *pas* de conversion implicite.

Conversion de types



- Les fonctions de conversion sont de la forme `<type_destination>_of_<type_source>`, par exemple `string_of_float` (convertit un flottant en chaîne).
- Il n'y a *pas* de conversion implicite.
- L'affichage s'obtient avec `print_<type>` par exemple `print_string` (excepté pour les booléens).

Syntaxe et évaluation

- En OCaml, on parlera d'expressions conditionnelles (et pas d'instructions conditionnelles).

Syntaxe et évaluation

- En OCaml, on parlera d'expressions conditionnelles (et pas d'instructions conditionnelles).
- La syntaxe d'une expression conditionnelle est :

```
if expr_bool then expr1 else expr2;;
```

Syntaxe et évaluation

- En OCaml, on parlera d'expressions conditionnelles (et pas d'instructions conditionnelles).
- La syntaxe d'une expression conditionnelle est :

```
if expr_bool then expr1 else expr2;;
```

- Cette expression est évaluée de la façon suivante :

Syntaxe et évaluation

- En OCaml, on parlera d'expressions conditionnelles (et pas d'instructions conditionnelles).
- La syntaxe d'une expression conditionnelle est :

```
if expr_bool then expr1 else expr2;;
```

- Cette expression est évaluée de la façon suivante :
  - On évalue `expr_bool`

Syntaxe et évaluation

- En OCaml, on parlera d'expressions conditionnelles (et pas d'instructions conditionnelles).
- La syntaxe d'une expression conditionnelle est :

```
if expr_bool then expr1 else expr2;;
```

- Cette expression est évaluée de la façon suivante :
  - On évalue `expr_bool`
  - Si le résultat est vrai, alors la valeur de l'expression conditionnelle est celle d'`expr1`

Syntaxe et évaluation

- En OCaml, on parlera d'expressions conditionnelles (et pas d'instructions conditionnelles).
- La syntaxe d'une expression conditionnelle est :

```
if expr_bool then expr1 else expr2;;
```

- Cette expression est évaluée de la façon suivante :
  - On évalue `expr_bool`
  - Si le résultat est vrai, alors la valeur de l'expression conditionnelle est celle d'`expr1`
  - Sinon, c'est celle d'`expr2`

Syntaxe et évaluation

- En OCaml, on parlera d'expressions conditionnelles (et pas d'instructions conditionnelles).
- La syntaxe d'une expression conditionnelle est :

```
if expr_bool then expr1 else expr2;;
```

- Cette expression est évaluée de la façon suivante :
  - On évalue `expr_bool`
  - Si le résultat est vrai, alors la valeur de l'expression conditionnelle est celle d'`expr1`
  - Sinon, c'est celle d'`expr2`
- `expr1` et `expr2` doivent toujours être du même type.

Syntaxe et évaluation

- En OCaml, on parlera d'expressions conditionnelles (et pas d'instructions conditionnelles).
- La syntaxe d'une expression conditionnelle est :

```
if expr_bool then expr1 else expr2;;
```

- Cette expression est évaluée de la façon suivante :
  - On évalue `expr_bool`
  - Si le résultat est vrai, alors la valeur de l'expression conditionnelle est celle d'`expr1`
  - Sinon, c'est celle d'`expr2`
- `expr1` et `expr2` doivent toujours être du même type.
- On doit donc toujours écrire la clause `else`

Syntaxe et évaluation

- En OCaml, on parlera d'expressions conditionnelles (et pas d'instructions conditionnelles).
- La syntaxe d'une expression conditionnelle est :

```
if expr_bool then expr1 else expr2;;
```

- Cette expression est évaluée de la façon suivante :
  - On évalue `expr_bool`
  - Si le résultat est vrai, alors la valeur de l'expression conditionnelle est celle d'`expr1`
  - Sinon, c'est celle d'`expr2`
- `expr1` et `expr2` doivent toujours être du même type.
- On doit donc toujours écrire la clause `else`

  Exception : lorsque `expr1` est de type `unit`, la clause `else` peut être omise et vaut par défaut `()`.

Exemple

1. Écrire une fonction `abs_entier` qui renvoie la valeur absolue de l'entier donné en argument.

Exemple

1. Écrire une fonction `abs_entier` qui renvoie la valeur absolue de l'entier donné en argument.

```
1  let abs_entier n =
2    if n <0 then -n else n;;
```

Exemple

1. Écrire une fonction `abs_entier` qui renvoie la valeur absolue de l'entier donné en argument.

```
1  let abs_entier n =
2    if n <0 then -n else n;;
```

2. Écrire une fonction `abs_flottant` qui renvoie la valeur absolue du flottant donné en argument.

Exemple

1. Écrire une fonction `abs_entier` qui renvoie la valeur absolue de l'entier donné en argument.

```
let abs_entier n =
  if n <0 then -n else n;;
```

2. Écrire une fonction `abs_flottant` qui renvoie la valeur absolue du flottant donné en argument.

```
let abs_flottant n =
  if n < 0. then -.n else n;;
```

Exemple

1. Écrire une fonction `abs_entier` qui renvoie la valeur absolue de l'entier donné en argument.

```
let abs_entier n =
  if n <0 then -n else n;;
```

2. Écrire une fonction `abs_flottant` qui renvoie la valeur absolue du flottant donné en argument.

```
let abs_flottant n =
  if n < 0. then -.n else n;;
```

3. Évaluer l'expression suivante et donner la valeur de `b` si `a` vaut `-3` :
`let b = 3 * if a<0 then 2 else 5;;`

Exemple

1. Écrire une fonction `abs_entier` qui renvoie la valeur absolue de l'entier donné en argument.

```
let abs_entier n =
  if n <0 then -n else n;;
```

2. Écrire une fonction `abs_flottant` qui renvoie la valeur absolue du flottant donné en argument.

```
let abs_flottant n =
  if n < 0. then -.n else n;;
```

3. Évaluer l'expression suivante et donner la valeur de b si a vaut -3 :
   `let b = 3 * if a<0 then 2 else 5;;`
   La valeur de b est 6.

Structure générale d'un programme OCaml

- Un programme OCaml repose sur l'évaluation d'une ou plusieurs expressions :

```
expr1 ;;
expr2 ;;
...
```

Structure générale d'un programme OCaml

- Un programme OCaml repose sur l'évaluation d'une ou plusieurs expressions :

```
expr1 ;;
expr2 ;;
...
```

- Les expressions sont évaluées dans un environnement (qui lie des identificateurs à des valeurs)

  `a + b` est une expression qui sera évaluée à 10 si `a` est lié à la valeur 4 et `b` à la valeur 6.

Structure générale d'un programme OCaml

- Un programme OCaml repose sur l'évaluation d'une ou plusieurs expressions :
  ```
  expr1 ;;
  expr2 ;;
  ...
  ```
- Les expressions sont évaluées dans un environnement (qui lie des identificateurs à des valeurs)
  `a + b` est une expression qui sera évaluée à 10 si `a` est lié à la valeur 4 et `b` à la valeur 6.
- On manipule l'environnement global grâce au mot-clé `let` :
  ```
  let id = expr;;
  ```

Structure générale d'un programme OCaml

- Un programme OCaml repose sur l'évaluation d'une ou plusieurs expressions :
```
expr1 ;;
expr2 ;;
...
```
- Les expressions sont évaluées dans un environnement (qui lie des identificateurs à des valeurs)
  `a + b` est une expression qui sera évaluée à 10 si `a` est lié à la valeur 4 et `b` à la valeur 6.
- On manipule l'environnement global grâce au mot-clé `let` :
```
let id = expr;;
```
- Le mot-clé `in` permet de créer un environnement local :
```
let id = expr1 in expr2;;
```
  Si `id` existe déjà dans l'environnement courant, il est provisoirement masqué (phénomène d'ombrage ou *shadowing*).

Fonctions

- L'appel d'une fonction `f` à n arguments s'écrit en OCaml : `f x1 x2 ... xn`

  ⚠ Attention au parenthésage !

  `f 2 * 3` est équivalent à `(f 2) * 3` et vaut $f(2) \times 3$

  `f (2*3)` vaut $f(2 \times 3)$.

Fonctions

- L'appel d'une fonction `f` à n arguments s'écrit en OCaml : `f x1 x2 ... xn`
  ⚠ Attention au parenthésage !
  `f 2 * 3` est équivalent à `(f 2) * 3` et vaut $f(2) \times 3$
  `f (2*3)` vaut $f(2 \times 3)$.
- OCaml traite les fonctions à plusieurs variables comme une fonction à un argument qui renvoie une fonction sur le reste des variables. C'est ce qu'on appelle la curryfication (du nom du mathématicien américain Haskell Curry). C'est-à-dire que, par exemple :

$$\begin{array}{rccc} f: & E \times F & \to & G \\ & (x,y) & \mapsto & z \end{array} \quad \text{s'interprète comme} \quad \begin{array}{rccc} f: & E & \to & \mathcal{F}(F,G) \\ & x & \mapsto & (y \mapsto z) \end{array}$$

Fonctions

- L'appel d'une fonction `f` à n arguments s'écrit en OCaml : `f x1 x2 ... xn`
  ⚠ Attention au parenthésage !
  `f 2 * 3` est équivalent à `(f 2) * 3` et vaut $f(2) \times 3$
  `f (2*3)` vaut $f(2 \times 3)$.
- OCaml traite les fonctions à plusieurs variables comme une fonction à un argument qui renvoie une fonction sur le reste des variables. C'est ce qu'on appelle la curryfication (du nom du mathématicien américain Haskell Curry). C'est-à-dire que, par exemple :
  $$\begin{array}{rccc} f: & E \times F & \to & G \\ & (x,y) & \mapsto & z \end{array} \text{ s'interprète comme } \begin{array}{rccc} f: & E & \to & \mathcal{F}(F,G) \\ & x & \mapsto & (y \mapsto z) \end{array}$$
- On donne donc la signature d'une fonction en OCaml, en listant les types des arguments séparés par -> et ensuite le type du résultat.

Fonctions

- L'appel d'une fonction `f` à n arguments s'écrit en OCaml : `f x1 x2 ... xn`
  ⚠ Attention au parenthésage !
  `f 2 * 3` est équivalent à `(f 2) * 3` et vaut $f(2) \times 3$
  `f (2*3)` vaut $f(2 \times 3)$.
- OCaml traite les fonctions à plusieurs variables comme une fonction à un argument qui renvoie une fonction sur le reste des variables. C'est ce qu'on appelle la curryfication (du nom du mathématicien américain Haskell Curry). C'est-à-dire que, par exemple :
  $$\begin{array}{rccc} f: & E \times F & \to & G \\ & (x,y) & \mapsto & z \end{array}$$ s'interprète comme $$\begin{array}{rccc} f: & E & \to & \mathcal{F}(F,G) \\ & x & \mapsto & (y \mapsto z) \end{array}$$
- On donne donc la signature d'une fonction en OCaml, en listant les types des arguments séparés par `->` et ensuite le type du résultat.
  Par exemple, une fonction `min3` renvoyant le minimum de trois entiers s'écrit `min3 : int -> int -> int -> int`.

Fonctions

- L'appel d'une fonction `f` à n arguments s'écrit en OCaml : `f x1 x2 ... xn`
  ⚠ Attention au parenthésage !
  `f 2 * 3` est équivalent à `(f 2) * 3` et vaut $f(2) \times 3$
  `f (2*3)` vaut $f(2 \times 3)$.
- OCaml traite les fonctions à plusieurs variables comme une fonction à un argument qui renvoie une fonction sur le reste des variables. C'est ce qu'on appelle la curryfication (du nom du mathématicien américain Haskell Curry). C'est-à-dire que, par exemple :
  $$\begin{array}{rccc} f: & E \times F & \to & G \\ & (x,y) & \mapsto & z \end{array}$$ s'interprète comme $$\begin{array}{rccc} f: & E & \to & \mathcal{F}(F,G) \\ & x & \mapsto & (y \mapsto z) \end{array}$$
- On donne donc la signature d'une fonction en OCaml, en listant les types des arguments séparés par `->` et ensuite le type du résultat.
  Par exemple, une fonction `min3` renvoyant le minimum de trois entiers s'écrit `min3 : int -> int -> int -> int`.
- L'application partielle d'une fonction (sans fournir tous ses arguments) renvoie une fonction.

Exemple

```
1  let aire_rectangle long larg = long * larg;;
2
3  let aire_cercle r =
4    let pi = 3.1415 in pi *. r *. r;;
5
6  print_int (aire_rectangle 4 10) ;;
7  print_newline() ;;
8  print_float (aire_cercle 5.);;
9  print_newline() ;;
```

Exemple

```
let distance x1 y1 x2 y2 =
  sqrt((x1-.x2)**2. +. (y1-.y2)**2.) ;;

(* Curryfication *)
let distance_a_zero = distance 0. 0. ;;

print_float (distance 0. 0. 10. 7.);;
print_newline();;
print_float (distance_a_zero 10. 7.);;
print_newline();;
```

Fonctions anonymes

- En OCaml, une fonction peut-être passée en argument ou renvoyée par une autre fonction, comme n'importe quelle autre type de données, on dit que les fonctions sont des *objets de première classe*.

Fonctions anonymes

- En OCaml, une fonction peut-être passée en argument ou renvoyée par une autre fonction, comme n'importe quelle autre type de données, on dit que les fonctions sont des *objets de première classe*.
- Le mot-clé `fun` permet de définir une fonction sans lui donner de nom :

```
fun p1 p2 ... -> expression
```

C'est l'analogue de la notation mathématique $(p_1, p_2, \dots) \mapsto$ expression.

Fonctions anonymes

- En OCaml, une fonction peut-être passée en argument ou renvoyée par une autre fonction, comme n'importe quelle autre type de données, on dit que les fonctions sont des *objets de première classe*.
- Le mot-clé `fun` permet de définir une fonction sans lui donner de nom :

  ```
  fun p1 p2 ... -> expression
  ```

  C'est l'analogue de la notation mathématique $(p_1, p_2, \dots) \mapsto$ expression.
- La définition habituelle d'une fonction avec `let` n'est qu'un raccourci :

  `let f x = expr` est syntaxiquement équivalent à `let f = fun x -> expr`

Fonctions anonymes

- En OCaml, une fonction peut-être passée en argument ou renvoyée par une autre fonction, comme n'importe quelle autre type de données, on dit que les fonctions sont des *objets de première classe*.
- Le mot-clé `fun` permet de définir une fonction sans lui donner de nom :

  ```
  fun p1 p2 ... -> expression
  ```

  C'est l'analogue de la notation mathématique $(p_1, p_2, \dots) \mapsto$ expression.
- La définition habituelle d'une fonction avec `let` n'est qu'un raccourci :

  `let f x = expr` est syntaxiquement équivalent à `let f = fun x -> expr`
- La curryfication s'interprète naturellement avec `fun` :

  `fun x y -> expr` est syntaxiquement équivalent à
  `fun x -> (fun y -> expr)`

Fonctions anonymes

- En OCaml, une fonction peut-être passée en argument ou renvoyée par une autre fonction, comme n'importe quelle autre type de données, on dit que les fonctions sont des *objets de première classe*.
- Le mot-clé `fun` permet de définir une fonction sans lui donner de nom :

  ```
  fun p1 p2 ... -> expression
  ```

  C'est l'analogue de la notation mathématique $(p_1, p_2, \dots) \mapsto$ expression.
- La définition habituelle d'une fonction avec `let` n'est qu'un raccourci :

  `let f x = expr` est syntaxiquement équivalent à `let f = fun x -> expr`
- La curryfication s'interprète naturellement avec `fun` :

  `fun x y -> expr` est syntaxiquement équivalent à
  `fun x -> (fun y -> expr)`
- On peut évaluer directement une fonction anonyme en la parenthésant :

  `(fun x -> x * x) 5` s'évalue à `25`

Fonctions d'ordre supérieur

- Une fonction d'ordre supérieur est une fonction qui prend une fonction en argument ou qui renvoie une fonction comme résultat.

Fonctions d'ordre supérieur

- Une fonction d'ordre supérieur est une fonction qui prend une fonction en argument ou qui renvoie une fonction comme résultat.
- Fonction renvoyant une fonction : on prend en entrée un entier `n` et on renvoie la fonction $x \mapsto x^n$:

```
let puissance n = fun x -> x ** (float_of_int n);;
```

Fonctions d'ordre supérieur

- Une fonction d'ordre supérieur est une fonction qui prend une fonction en argument ou qui renvoie une fonction comme résultat.
- Fonction renvoyant une fonction : on prend en entrée un entier `n` et on renvoie la fonction $x \mapsto x^n$:

```
let puissance n = fun x -> x ** (float_of_int n);;
```

Exemple : `let carre = puissance 2` renvoie la fonction carrée (sur les flottants).

Fonctions d'ordre supérieur

- Une fonction d'ordre supérieur est une fonction qui prend une fonction en argument ou qui renvoie une fonction comme résultat.
- Fonction renvoyant une fonction : on prend en entrée un entier `n` et on renvoie la fonction $x \mapsto x^n$:

```
let puissance n = fun x -> x ** (float_of_int n);;
```

Exemple : `let carre = puissance 2` renvoie la fonction carrée (sur les flottants).

- Fonction prenant une fonction en argument :

```
let frondf f x = f (f x);;
```

Fonctions d'ordre supérieur

- Une fonction d'ordre supérieur est une fonction qui prend une fonction en argument ou qui renvoie une fonction comme résultat.
- Fonction renvoyant une fonction : on prend en entrée un entier `n` et on renvoie la fonction $x \mapsto x^n$:

```
let puissance n = fun x -> x ** (float_of_int n);;
```

Exemple : `let carre = puissance 2` renvoie la fonction carrée (sur les flottants).

- Fonction prenant une fonction en argument :

```
let frondf f x = f (f x);;
```

Exemple : `frondf (fun x -> 2 * x) 3` s'évalue en 12.

Fonctions d'ordre supérieur

- Une fonction d'ordre supérieur est une fonction qui prend une fonction en argument ou qui renvoie une fonction comme résultat.
- Fonction renvoyant une fonction : on prend en entrée un entier `n` et on renvoie la fonction $x \mapsto x^n$:

```
let puissance n = fun x -> x ** (float_of_int n);;
```

Exemple : `let carre = puissance 2` renvoie la fonction carrée (sur les flottants).

- Fonction prenant une fonction en argument :

```
let frondf f x = f (f x);;
```

Exemple : `frondf (fun x -> 2 * x) 3` s'évalue en 12.

- Intérêt des fonctions anonymes : elles permettent de définir localement un traitement

Fonctions d'ordre supérieur

- Une fonction d'ordre supérieur est une fonction qui prend une fonction en argument ou qui renvoie une fonction comme résultat.
- Fonction renvoyant une fonction : on prend en entrée un entier `n` et on renvoie la fonction $x \mapsto x^n$:

```
let puissance n = fun x -> x ** (float_of_int n);;
```

Exemple : `let carre = puissance 2` renvoie la fonction carrée (sur les flottants).

- Fonction prenant une fonction en argument :

```
let frondf f x = f (f x);;
```

Exemple : `frondf (fun x -> 2 * x) 3` s'évalue en 12.

- Intérêt des fonctions anonymes : elles permettent de définir localement un traitement

Exemple : `List.map (fun x -> x * x) [1; 2; 3]` s'évalue en `[1; 4; 9]`.

Principe

Les trois mécanismes suivants permettent de construire de nouveaux types en OCaml à partir des types de base (`int`, `float`, `bool`, ...)

Principe

Les trois mécanismes suivants permettent de construire de nouveaux types en OCaml à partir des types de base (`int`, `float`, `bool`, ...)

- Les couples, triplets et plus généralement les n-uplets, définissent des types de la forme $(x_1, \ldots, x_n)$ où chacun des x_i peut avoir son propre type de base.

Principe

Les trois mécanismes suivants permettent de construire de nouveaux types en OCaml à partir des types de base (`int`, `float`, `bool`, ...)

- Les couples, triplets et plus généralement les n-uplets, définissent des types de la forme $(x_1, \ldots, x_n)$ où chacun des x_i peut avoir son propre type de base.
  Ex : `type point = float * float;;` définit un couple de flottants.

Principe

Les trois mécanismes suivants permettent de construire de nouveaux types en OCaml à partir des types de base (`int`, `float`, `bool`, ...)

- Les couples, triplets et plus généralement les n-uplets, définissent des types de la forme $(x_1, \ldots, x_n)$ où chacun des x_i peut avoir son propre type de base.
  Ex : `type point = float * float;;` définit un couple de flottants.
  Les fonctions `fst` et `snd` permettent d'accéder au premier et au second élément d'un couple (2-uplet). Pour les n-uplets ($n \geq 3$), on utilise le filtrage ou la destructuration `let (x, y, z) = t`.

Principe

Les trois mécanismes suivants permettent de construire de nouveaux types en OCaml à partir des types de base (`int`, `float`, `bool`, ...)

- Les couples, triplets et plus généralement les n-uplets, définissent des types de la forme $(x_1, \ldots, x_n)$ où chacun des x_i peut avoir son propre type de base.
  Ex : `type point = float * float;;` définit un couple de flottants.
  Les fonctions `fst` et `snd` permettent d'accéder au premier et au second élément d'un couple (2-uplet). Pour les n-uplets ($n \geq 3$), on utilise le filtrage ou la destructuration `let (x, y, z) = t`.
- Les types enregistrements (ou types produits étiquetés).

Principe

Les trois mécanismes suivants permettent de construire de nouveaux types en OCaml à partir des types de base (`int`, `float`, `bool`, ...)

- Les couples, triplets et plus généralement les n-uplets, définissent des types de la forme $(x_1, \ldots, x_n)$ où chacun des x_i peut avoir son propre type de base.
  Ex : `type point = float * float;;` définit un couple de flottants.
  Les fonctions `fst` et `snd` permettent d'accéder au premier et au second élément d'un couple (2-uplet). Pour les n-uplets ($n \geq 3$), on utilise le filtrage ou la destructuration `let (x, y, z) = t`.
- Les types enregistrements (ou types produits étiquetés).
  Ex : `type date = {jour : int; mois : string; annee : int}`

Principe

Les trois mécanismes suivants permettent de construire de nouveaux types en OCaml à partir des types de base (`int`, `float`, `bool`, ...)

- Les couples, triplets et plus généralement les n-uplets, définissent des types de la forme $(x_1, \ldots, x_n)$ où chacun des x_i peut avoir son propre type de base.
  Ex : `type point = float * float;;` définit un couple de flottants.
  Les fonctions `fst` et `snd` permettent d'accéder au premier et au second élément d'un couple (2-uplet). Pour les n-uplets ($n \geq 3$), on utilise le filtrage ou la destructuration `let (x, y, z) = t`.
- Les types enregistrements (ou types produits étiquetés).
  Ex : `type date = {jour : int; mois : string; annee : int}`
  Accès aux champs avec la notation `.` (comme en C).

Principe

Les trois mécanismes suivants permettent de construire de nouveaux types en OCaml à partir des types de base (`int`, `float`, `bool`, ...)

- Les couples, triplets et plus généralement les n-uplets, définissent des types de la forme $(x_1, \ldots, x_n)$ où chacun des x_i peut avoir son propre type de base.
  Ex : `type point = float * float;;` définit un couple de flottants.
  Les fonctions `fst` et `snd` permettent d'accéder au premier et au second élément d'un couple (2-uplet). Pour les n-uplets ($n \geq 3$), on utilise le filtrage ou la destructuration `let (x, y, z) = t`.
- Les types enregistrements (ou types produits étiquetés).
  Ex : `type date = {jour : int; mois : string; annee : int}`
  Accès aux champs avec la notation `.` (comme en C).
- Les types unions (ou encore types sommes) où on liste les valeurs possibles séparées par des `|`.

Principe

Les trois mécanismes suivants permettent de construire de nouveaux types en OCaml à partir des types de base (`int`, `float`, `bool`, ...)

- Les couples, triplets et plus généralement les n-uplets, définissent des types de la forme $(x_1, \ldots, x_n)$ où chacun des x_i peut avoir son propre type de base.
  Ex : `type point = float * float;;` définit un couple de flottants.
  Les fonctions `fst` et `snd` permettent d'accéder au premier et au second élément d'un couple (2-uplet). Pour les n-uplets ($n \geq 3$), on utilise le filtrage ou la destructuration `let (x, y, z) = t`.
- Les types enregistrements (ou types produits étiquetés).
  Ex : `type date = {jour : int; mois : string; annee : int}`
  Accès aux champs avec la notation `.` (comme en C).
- Les types unions (ou encore types sommes) où on liste les valeurs possibles séparées par des `|`.
  Ex : `type signe = Positif | Negatif | Nul;;`

Principe

Les trois mécanismes suivants permettent de construire de nouveaux types en OCaml à partir des types de base (`int`, `float`, `bool`, ...)

- Les couples, triplets et plus généralement les n-uplets, définissent des types de la forme $(x_1, \ldots, x_n)$ où chacun des x_i peut avoir son propre type de base.
  Ex : `type point = float * float;;` définit un couple de flottants.
  Les fonctions `fst` et `snd` permettent d'accéder au premier et au second élément d'un couple (2-uplet). Pour les n-uplets ($n \geq 3$), on utilise le filtrage ou la destructuration `let (x, y, z) = t`.
- Les types enregistrements (ou types produits étiquetés).
  Ex : `type date = {jour : int; mois : string; annee : int}`
  Accès aux champs avec la notation `.` (comme en C).
- Les types unions (ou encore types sommes) où on liste les valeurs possibles séparées par des `|`.
  Ex : `type signe = Positif | Negatif | Nul;;`
  Ex : `type carte = Roi | Dame | Valet | Nombre of int;;`

Principe

Les trois mécanismes suivants permettent de construire de nouveaux types en OCaml à partir des types de base (`int`, `float`, `bool`, ...)

- Les couples, triplets et plus généralement les n-uplets, définissent des types de la forme $(x_1, \ldots, x_n)$ où chacun des x_i peut avoir son propre type de base.
  Ex : `type point = float * float;;` définit un couple de flottants.
  Les fonctions `fst` et `snd` permettent d'accéder au premier et au second élément d'un couple (2-uplet). Pour les n-uplets ($n \geq 3$), on utilise le filtrage ou la destructuration `let (x, y, z) = t`.
- Les types enregistrements (ou types produits étiquetés).
  Ex : `type date = {jour : int; mois : string; annee : int}`
  Accès aux champs avec la notation `.` (comme en C).
- Les types unions (ou encore types sommes) où on liste les valeurs possibles séparées par des `|`.
  Ex : `type signe = Positif | Negatif | Nul;;`
  Ex : `type carte = Roi | Dame | Valet | Nombre of int;;`
  ⚠ Les constructeurs commencent par une majuscule.

Exemple

1. Définir le type complexe comme un couple de flottants, écrire la fonction module pour ce type.

Exemple

1. Définir le type complexe comme un couple de flottants, écrire la fonction module pour ce type.
2. Définir le type menu comme un type enregistrement composé des champs `entree` (string), `plat` (string), `dessert` (string) et `prix` (float).

Exemple

1. Définir le type complexe comme un couple de flottants, écrire la fonction module pour ce type.
2. Définir le type menu comme un type enregistrement composé des champs `entree` (string), `plat` (string), `dessert` (string) et `prix` (float).
3. Définir le type rbarre ($\overline{\mathbb{R}}$), comme un type somme pouvant être `Plusinfini`, `Moinsinfini` et les `flottants`.

Exemple : correction

1. Type produit complexe

```
1  type complexe = float * float;;
2
3  let modu (z:complexe) =
4    let x = (fst z) in
5      let y = (snd z) in
6        sqrt(x**2.+.y**2.);;
```

⚠ On utilise l'annotation de type pour spécifier que l'argument est de type `complexe`.

Exemple : correction

1. Type produit complexe

```
1  type complexe = float * float;;
2
3  let modu (z:complexe) =
4    let x = (fst z) in
5      let y = (snd z) in
6        sqrt(x**2.+.y**2.);;
```

⚠ On utilise l'annotation de type pour spécifier que l'argument est de type `complexe`.

2. Type menu enregistrement

```
1  type menu = {entree : string; plat : string; dessert : string;
   ↪  prix : float};;
```

Exemple : correction

1. Type produit complexe

```
1  type complexe = float * float;;
2
3  let modu (z:complexe) =
4    let x = (fst z) in
5      let y = (snd z) in
6        sqrt(x**2.+.y**2.);;
```

⚠ On utilise l'annotation de type pour spécifier que l'argument est de type `complexe`.

2. Type menu enregistrement

```
1  type menu = {entree : string; plat : string; dessert : string;
   ↪  prix : float};;
```

3. Type somme rbarre

```
1  type rbarre = Plusinfini | Moinsinfini | Nombre of float;;
```

Définition

Définition

- Le filtrage par motif (*pattern matching*) est un mécanisme permettant de travailler efficacement sur les types construits en les décomposant et les analysant suivant leur structure. On peut ainsi indiquer l'expression à évaluer suivant la forme spécifique de l'entrée.

Définition

- Le filtrage par motif (*pattern matching*) est un mécanisme permettant de travailler efficacement sur les types construits en les décomposant et les analysant suivant leur structure. On peut ainsi indiquer l'expression à évaluer suivant la forme spécifique de l'entrée.
- La syntaxe générale est :

```
match expr with
| motif1 -> expr1
| ⋮
| motifn -> exprn
```

Définition

- Le filtrage par motif (*pattern matching*) est un mécanisme permettant de travailler efficacement sur les types construits en les décomposant et les analysant suivant leur structure. On peut ainsi indiquer l'expression à évaluer suivant la forme spécifique de l'entrée.
- La syntaxe générale est :

```
match expr with
| motif1 -> expr1
| ⋮
| motifn -> exprn
```

⚠ Le filtrage doit être exhaustif, le caractère spécial _ indique un motif qui correspond à toutes les entrées.

Définition

- Le filtrage par motif (*pattern matching*) est un mécanisme permettant de travailler efficacement sur les types construits en les décomposant et les analysant suivant leur structure. On peut ainsi indiquer l'expression à évaluer suivant la forme spécifique de l'entrée.
- La syntaxe générale est :

```
match expr with
| motif1 -> expr1
| ⋮
| motifn -> exprn
```

⚠ Le filtrage doit être exhaustif, le caractère spécial _ indique un motif qui correspond à toutes les entrées.

- L'ordre du filtrage est important car si deux motifs correspondent à une entrée, c'est l'expression du premier rencontré qui sera évaluée.

Définition

- Le filtrage par motif (*pattern matching*) est un mécanisme permettant de travailler efficacement sur les types construits en les décomposant et les analysant suivant leur structure. On peut ainsi indiquer l'expression à évaluer suivant la forme spécifique de l'entrée.
- La syntaxe générale est :

```
match expr with
| motif1 -> expr1
| ⋮
| motifn -> exprn
```

⚠ Le filtrage doit être exhaustif, le caractère spécial _ indique un motif qui correspond à toutes les entrées.

- L'ordre du filtrage est important car si deux motifs correspondent à une entrée, c'est l'expression du premier rencontré qui sera évaluée.
- Chaque identifiant apparaît une fois au plus dans un motif (linéarité des motifs).

Définition

- Le filtrage par motif (*pattern matching*) est un mécanisme permettant de travailler efficacement sur les types construits en les décomposant et les analysant suivant leur structure. On peut ainsi indiquer l'expression à évaluer suivant la forme spécifique de l'entrée.
- La syntaxe générale est :

```
match expr with
| motif1 -> expr1
| ⋮
| motifn -> exprn
```

⚠ Le filtrage doit être exhaustif, le caractère spécial _ indique un motif qui correspond à toutes les entrées.

- L'ordre du filtrage est important car si deux motifs correspondent à une entrée, c'est l'expression du premier rencontré qui sera évaluée.
- Chaque identifiant apparaît une fois au plus dans un motif (linéarité des motifs).

Exemple

On a déjà rencontré le type carte :

```
type carte = As | Roi | Dame | Valet | Nombre of int;;
```

On peut associer chaque carte à sa valeur (à la belote) à l'aide d'un pattern matching :

```
1  type carte = As | Roi | Dame | Valet | Nombre of int;;
2
3  let valeur c =
4    match c with
5    | As -> 11
6    | Roi -> 4
7    | Dame -> 3
8    | Valet -> 2
9    | Nombre 10 -> 10
10   | _ -> 0
```

Exemple

On peut aussi filtrer un couple de cartes afin de détecter une éventuelle paire (deux cartes identiques) :

```
let est_paire c1 c2 =
  match (c1, c2) with
  | (As, As) -> true
  | (Roi, Roi) -> true
  | (Dame, Dame) -> true
  | (Valet, Valet) -> true
  | (Nombre x, Nombre y) -> x = y
  | (_,_) -> false
```

Exercice

1. Définir un type somme `signe` avec les valeurs `Negatif`, `Positif` et `Nul`.

Exercice

1. Définir un type somme `signe` avec les valeurs `Negatif`, `Positif` et `Nul`.

```
1    type signe = Negatif | Nul | Positif
```

Exercice

1. Définir un type somme `signe` avec les valeurs `Negatif`, `Positif` et `Nul`.

```
1    type signe = Negatif | Nul | Positif
```

2. Écrire une fonction `produit` qui renvoie le signe d'un produit.

Exercice

1. Définir un type somme `signe` avec les valeurs `Negatif`, `Positif` et `Nul`.

```
type signe = Negatif | Nul | Positif
```

2. Écrire une fonction `produit` qui renvoie le signe d'un produit.

```
let produit s1 s2 =
  match s1, s2 with
  | Nul, _ -> Nul
  | _, Nul -> Nul
  | Positif, Positif -> Positif
  | Negatif, Negatif -> Positif
  | _,_ -> Negatif
```

Exemple

```
1  let deux_egaux a b c =
2    (a = b || a=c || b=c)
```

Exemple

```
1  let deux_egaux a b c =
2    (a = b || a=c || b=c)
```

Cette fonction peut prendre en entrée trois entiers, trois flottants, trois caractères, ... Sa signature est `deux_egaux : 'a -> 'a -> 'a -> bool`. On dit qu'elle est polymorphe car elle accepte des entrées de types différents, le type des entrées est donc noté par OCaml `'a`.

Exemple

```
1  let deux_egaux a b c =
2    (a = b || a=c || b=c)
```

Cette fonction peut prendre en entrée trois entiers, trois flottants, trois caractères, ... Sa signature est `deux_egaux : 'a -> 'a -> 'a -> bool`. On dit qu'elle est polymorphe car elle accepte des entrées de types différents, le type des entrées est donc noté par OCaml `'a`.

Définition

Le polymorphisme est un mécanisme permettant d'écrire des fonctions prenant en entrée des valeurs de types différents. Le type des arguments est alors noté `'a`, `'b`, ...

Le type `list`

- Le type `list` est prédéfini en OCaml et permet de représenter des listes de valeurs de même type. Une liste est immuable (non modifiable).

Le type `list`

- Le type `list` est prédéfini en OCaml et permet de représenter des listes de valeurs de même type. Une liste est immuable (non modifiable).
- On peut définir une liste en mettant les éléments entre crochets et en les séparant par des points-virgules.

Le type `list`

- Le type `list` est prédéfini en OCaml et permet de représenter des listes de valeurs de même type. Une liste est immuable (non modifiable).
- On peut définir une liste en mettant les éléments entre crochets et en les séparant par des points-virgules.
  `let p = [2; 3; 5; 7]` : une liste d'entiers

Le type `list`

- Le type `list` est prédéfini en OCaml et permet de représenter des listes de valeurs de même type. Une liste est immuable (non modifiable).
- On peut définir une liste en mettant les éléments entre crochets et en les séparant par des points-virgules.
  `let p = [2; 3; 5; 7]` : une liste d'entiers
  `let lang = ["Python"; "Ada"; "C"]` : une liste de chaînes de caractères

Le type `list`

- Le type `list` est prédéfini en OCaml et permet de représenter des listes de valeurs de même type. Une liste est immuable (non modifiable).
- On peut définir une liste en mettant les éléments entre crochets et en les séparant par des points-virgules.
  `let p = [2; 3; 5; 7]` : une liste d'entiers
  `let lang = ["Python"; "Ada"; "C"]` : une liste de chaînes de caractères
  `let bug = [3.14; 2.71; 42; 0.57]` : erreur car liste non homogène

Le type `list`

- Le type `list` est prédéfini en OCaml et permet de représenter des listes de valeurs de même type. Une liste est immuable (non modifiable).
- On peut définir une liste en mettant les éléments entre crochets et en les séparant par des points-virgules.
  `let p = [2; 3; 5; 7]` : une liste d'entiers
  `let lang = ["Python"; "Ada"; "C"]` : une liste de chaînes de caractères
  `let bug = [3.14; 2.71; 42; 0.57]` : erreur car liste non homogène
- Une liste est un type récursif, en effet :

Le type `list`

- Le type `list` est prédéfini en OCaml et permet de représenter des listes de valeurs de même type. Une liste est immuable (non modifiable).
- On peut définir une liste en mettant les éléments entre crochets et en les séparant par des points-virgules.
  `let p = [2; 3; 5; 7]` : une liste d'entiers
  `let lang = ["Python"; "Ada"; "C"]` : une liste de chaînes de caractères
  `let bug = [3.14; 2.71; 42; 0.57]` : erreur car liste non homogène
- Une liste est un type récursif, en effet :
  - elle est vide (notée `[]`) ou

Le type `list`

- Le type `list` est prédéfini en OCaml et permet de représenter des listes de valeurs de même type. Une liste est immuable (non modifiable).
- On peut définir une liste en mettant les éléments entre crochets et en les séparant par des points-virgules.
  `let p = [2; 3; 5; 7]` : une liste d'entiers
  `let lang = ["Python"; "Ada"; "C"]` : une liste de chaînes de caractères
  `let bug = [3.14; 2.71; 42; 0.57]` : erreur car liste non homogène
- Une liste est un type récursif, en effet :
  - elle est vide (notée `[]`) ou
  - elle est constituée d'un élément (appelé tête) et d'une liste (appelée queue).

Le type `list`

- Le type `list` est prédéfini en OCaml et permet de représenter des listes de valeurs de même type. Une liste est immuable (non modifiable).
- On peut définir une liste en mettant les éléments entre crochets et en les séparant par des points-virgules.
  `let p = [2; 3; 5; 7]` : une liste d'entiers
  `let lang = ["Python"; "Ada"; "C"]` : une liste de chaînes de caractères
  `let bug = [3.14; 2.71; 42; 0.57]` : erreur car liste non homogène
- Une liste est un type récursif, en effet :
  - elle est vide (notée `[]`) ou
  - elle est constituée d'un élément (appelé tête) et d'une liste (appelée queue).
- Le filtrage par motif est la technique standard pour opérer sur les listes. On utilise alors le constructeur `::` pour « séparer » la tête et la queue.

Exemple de filtrage par motif sur les listes

- Fonction qui renvoie la longueur d'une liste.

Exemple de filtrage par motif sur les listes

- Fonction qui renvoie la longueur d'une liste.

```
1  let rec taille liste =
2    match liste with
3    | [] -> 0
4    | h::t -> 1 + taille t
```

Exemple de filtrage par motif sur les listes

- Fonction qui renvoie la longueur d'une liste.

```
1  let rec taille liste =
2    match liste with
3    | [] -> 0
4    | h::t -> 1 + taille t
```

- Fonction qui renvoie `true` si un élément est dans une liste et `false` sinon.

Exemple de filtrage par motif sur les listes

- Fonction qui renvoie la longueur d'une liste.

```
let rec taille liste =
  match liste with
  | [] -> 0
  | h::t -> 1 + taille t
```

- Fonction qui renvoie `true` si un élément est dans une liste et `false` sinon.

```
let rec est_dans elt liste =
  match liste with
  | [] -> false
  | h::t -> elt=h || est_dans elt t
```

Fonctions usuelles sur les listes

- `@` est l'opérateur de concaténation.

Fonctions usuelles sur les listes

- `@` est l'opérateur de concaténation.
  Par exemple, `let exl = [1; -2] @ [-3; 4]` va créer la liste `exl = [1; -2; -3; 4]`.

Fonctions usuelles sur les listes

- `@` est l'opérateur de concaténation.
  Par exemple, `let exl = [1; -2] @ [-3; 4]` va créer la liste `exl = [1; -2; -3; 4]`.
- `List.hd` et `List.tl` permettent de récupérer respectivement la tête et la queue de la liste.

Fonctions usuelles sur les listes

- `@` est l'opérateur de concaténation.
  Par exemple, `let exl = [1; -2] @ [-3; 4]` va créer la liste `exl = [1; -2; -3; 4]`.
- `List.hd` et `List.tl` permettent de récupérer respectivement la tête et la queue de la liste.
  Par exemple, `List.hd exl` renvoie l'entier 1 et `List.tl exl` renvoie la liste `[-2; -3; 4]`

Fonctions usuelles sur les listes

- `@` est l'opérateur de concaténation.
  Par exemple, `let exl = [1; -2] @ [-3; 4]` va créer la liste `exl = [1; -2; -3; 4]`.
- `List.hd` et `List.tl` permettent de récupérer respectivement la tête et la queue de la liste.
  Par exemple, `List.hd exl` renvoie l'entier 1 et `List.tl exl` renvoie la liste `[-2; -3; 4]`
- `List.length` renvoie la longueur de la liste.

Fonctions usuelles sur les listes

- `@` est l'opérateur de concaténation.
  Par exemple, `let exl = [1; -2] @ [-3; 4]` va créer la liste `exl = [1; -2; -3; 4]`.
- `List.hd` et `List.tl` permettent de récupérer respectivement la tête et la queue de la liste.
  Par exemple, `List.hd exl` renvoie l'entier 1 et `List.tl exl` renvoie la liste `[-2; -3; 4]`
- `List.length` renvoie la longueur de la liste.
  Par exemple, `List.length exl` renvoie 4

Fonctions usuelles sur les listes

- `@` est l'opérateur de concaténation.
  Par exemple, `let exl = [1; -2] @ [-3; 4]` va créer la liste `exl = [1; -2; -3; 4]`.
- `List.hd` et `List.tl` permettent de récupérer respectivement la tête et la queue de la liste.
  Par exemple, `List.hd exl` renvoie l'entier 1 et `List.tl exl` renvoie la liste `[-2; -3; 4]`
- `List.length` renvoie la longueur de la liste.
  Par exemple, `List.length exl` renvoie 4
- `List.map` renvoie la liste obtenue en appliquant la fonction donnée en paramètre à chaque élément de la liste.

Fonctions usuelles sur les listes

- `@` est l'opérateur de concaténation.
  Par exemple, `let exl = [1; -2] @ [-3; 4]` va créer la liste `exl = [1; -2; -3; 4]`.
- `List.hd` et `List.tl` permettent de récupérer respectivement la tête et la queue de la liste.
  Par exemple, `List.hd exl` renvoie l'entier 1 et `List.tl exl` renvoie la liste `[-2; -3; 4]`
- `List.length` renvoie la longueur de la liste.
  Par exemple, `List.length exl` renvoie 4
- `List.map` renvoie la liste obtenue en appliquant la fonction donnée en paramètre à chaque élément de la liste.
  Par exemple, `List.map abs exl` renvoie `[1; 2; 3; 4]`

Fonctions usuelles sur les listes

- `@` est l'opérateur de concaténation.
  Par exemple, `let exl = [1; -2] @ [-3; 4]` va créer la liste `exl = [1; -2; -3; 4]`.
- `List.hd` et `List.tl` permettent de récupérer respectivement la tête et la queue de la liste.
  Par exemple, `List.hd exl` renvoie l'entier 1 et `List.tl exl` renvoie la liste `[-2; -3; 4]`
- `List.length` renvoie la longueur de la liste.
  Par exemple, `List.length exl` renvoie 4
- `List.map` renvoie la liste obtenue en appliquant la fonction donnée en paramètre à chaque élément de la liste.
  Par exemple, `List.map abs exl` renvoie `[1; 2; 3; 4]`
- `List.fold_left` et `List.fold_right` permettent d'effectuer des opérations entre les éléments d'une liste. On doit fournir l'opération et l'opérande de départ.

Fonctions usuelles sur les listes

- `@` est l'opérateur de concaténation.
  Par exemple, `let exl = [1; -2] @ [-3; 4]` va créer la liste `exl = [1; -2; -3; 4]`.
- `List.hd` et `List.tl` permettent de récupérer respectivement la tête et la queue de la liste.
  Par exemple, `List.hd exl` renvoie l'entier 1 et `List.tl exl` renvoie la liste `[-2; -3; 4]`
- `List.length` renvoie la longueur de la liste.
  Par exemple, `List.length exl` renvoie 4
- `List.map` renvoie la liste obtenue en appliquant la fonction donnée en paramètre à chaque élément de la liste.
  Par exemple, `List.map abs exl` renvoie `[1; 2; 3; 4]`
- `List.fold_left` et `List.fold_right` permettent d'effectuer des opérations entre les éléments d'une liste. On doit fournir l'opération et l'opérande de départ.
  Par exemple, `List.fold_left ( * ) 1 exl` renvoie 24.

Exemple

Supposons qu'on veuille écrire une fonction `premier_pair : int list -> int` qui renvoie le premier élément pair d'une liste d'entiers. On peut gérer le cas d'une liste ne contenant pas d'entiers pairs en renvoyant une valeur spéciale, par exemple `Aucun`. On doit donc créer un type somme pour désigner le type renvoyé :

Exemple

Supposons qu'on veuille écrire une fonction `premier_pair : int list -> int` qui renvoie le premier élément pair d'une liste d'entiers. On peut gérer le cas d'une liste ne contenant pas d'entiers pairs en renvoyant une valeur spéciale, par exemple `Aucun`. On doit donc créer un type somme pour désigner le type renvoyé :

```
1 type entier_aucun =
2 | Aucun
3 | Entier of int
```

Exemple

Supposons qu'on veuille écrire une fonction `premier_pair : int list -> int` qui renvoie le premier élément pair d'une liste d'entiers. On peut gérer le cas d'une liste ne contenant pas d'entiers pairs en renvoyant une valeur spéciale, par exemple `Aucun`. On doit donc créer un type somme pour désigner le type renvoyé :

```
1 type entier_aucun =
2 | Aucun
3 | Entier of int
```

Ce type est en réalité prédéfini et polymorphe en OCaml et s'appelle le type option (`'a option`), c'est-à-dire qu'il contient une valeur (`Some of 'a`) ou rien (`None`).

Exemple

Supposons qu'on veuille écrire une fonction `premier_pair : int list -> int` qui renvoie le premier élément pair d'une liste d'entiers. On peut gérer le cas d'une liste ne contenant pas d'entiers pairs en renvoyant une valeur spéciale, par exemple `Aucun`. On doit donc créer un type somme pour désigner le type renvoyé :

```
1 type entier_aucun =
2 | Aucun
3 | Entier of int
```

Ce type est en réalité prédéfini et polymorphe en OCaml et s'appelle le type option (`'a option`), c'est-à-dire qu'il contient une valeur (`Some of 'a`) ou rien (`None`).

```
1 let rec premier_pair l =
2 match l with
3 | [] -> None
4 | h::t -> if h mod 2 = 0 then Some h else premier_pair t;;
```

Exemple

Supposons qu'on veuille écrire une fonction `premier_pair : int list -> int` qui renvoie le premier élément pair d'une liste d'entiers. On peut gérer le cas d'une liste ne contenant pas d'entiers pairs en renvoyant une valeur spéciale, par exemple `Aucun`. On doit donc créer un type somme pour désigner le type renvoyé :

```
1  type entier_aucun =
2  | Aucun
3  | Entier of int
```

Ce type est en réalité prédéfini et polymorphe en OCaml et s'appelle le type option (`'a option`), c'est-à-dire qu'il contient une valeur (`Some of 'a`) ou rien (`None`).

```
1  let rec premier_pair l =
2  match l with
3  | [] -> None
4  | h::t -> if h mod 2 = 0 then Some h else premier_pair t;;
```

La fonction a pour type `premier_pair : int list -> int option`.